

O Conselho das Quatro Ferramentas: Extraindo o Máximo Desempenho de LLMs com Claude Projects, Claude Code, Google Antigravity e NotebookLM

Resumo Executivo (TL;DR)

- **A jogada disruptiva é uma arquitetura de “conselho” multi-fornecedor:** usar o NotebookLM (Gemini 3) como camada de conhecimento resistente a alucinação, o Claude Projects como memória estratégica persistente, o Claude Code (Opus 4.8) como executor de alta precisão, e o Google Antigravity (Gemini 3.5 Flash) como orquestrador/verificador paralelo — costurados pelo MCP, que foi doado em 9 de dezembro de 2025 à Agentic AI Foundation (um fundo dirigido sob a Linux Foundation, co-fundado com Block e OpenAI) e hoje abrange “mais de 10.000 servidores MCP publicados” com “mais de 97 milhões de downloads mensais de SDK.”
- **O MCP já é o tecido conectivo hoje:** pontes MCP da comunidade permitem que o Claude Code controle o NotebookLM (recuperação zero-alucinação, ancorada em citações, que economiza tokens do Claude) e permitem que Antigravity e Claude Code deleguem tarefas um ao outro como subagentes — ou seja, dá para construir um sistema de quatro ferramentas e duas famílias de modelos **agora**, sem esperar integrações oficiais.
- **Combine famílias de modelos por diversidade, não por redundância:** os próprios dados da Anthropic mostram que um sistema multi-agente com Claude Opus 4 como líder e subagentes Claude Sonnet 4 “superou o Claude Opus 4 single-agent em 90,2% na nossa avaliação interna de pesquisa”, e o padrão LLM Council de Karpathy mostra que a revisão cruzada entre modelos reduz a falha correlacionada — mas sistemas multi-agente “usam cerca de 15x mais tokens que chats”, então reserve o conselho completo para decisões de alto risco.

Principais Achados

1. **As quatro ferramentas convergiram para as mesmas primitivas em 2026** — bases de conhecimento persistentes, instruções customizadas, conectividade MCP, subagentes, skills e artefatos verificáveis — e é exatamente isso que as torna combináveis em um único sistema, em vez de quatro silos.

2. **As duas famílias de modelos são complementares, não intercambiáveis.** O Claude Opus 4.8 lidera em precisão de código e consistência em tarefas longas; o Gemini 3.5 Flash é cerca de 4× mais rápido e bem mais barato, sendo o motor certo para orquestração paralela e verificação em alto volume. Usar o modelo mais barato/rápido para abrangência e o modelo preciso para profundidade é o padrão dominante de controle de custo.
 3. **O NotebookLM é a arma secreta como “oráculo de ancoragem”** — porque ele se recusa a responder fora das suas fontes, funciona como um firewall de alucinação que outros agentes podem consultar via MCP, reduzindo drasticamente o problema de alucinação de API na codificação agêntica.
 4. **Padrões de “conselho de agentes” são reais e já estão em produção** — o LLM Council open-source de Karpathy, as skills `agent-council` e `llm-council` do Claude Code, e a pesquisa acadêmica de “Council Mode” demonstram a deliberação multi-modelo funcionando na prática.
 5. **O maior risco é a falha correlacionada e o custo.** Sistemas multi-agente usam ~15× os tokens de um único chat; conselhos cross-model somam ~2–3× de latência. São trade-offs deliberados, a serem usados seletivamente.
-

Parte 1 — Capacidades Brutas por Ferramenta

1. Claude Projects (claude.ai)

Capacidade central e o que o torna poderoso. Projects são espaços de trabalho autocontidos que reúnem três coisas em contexto persistente: uma base de conhecimento, instruções customizadas e históricos de chat agrupados. Resolvem o “imposto do cold start” — você nunca reexplica quem é nem reenvia documentos de referência. A base de conhecimento usa o Contextual Retriever da Anthropic (RAG); em planos pagos, quando o conhecimento do projeto se aproxima da janela de contexto, o Claude habilita automaticamente o modo RAG, expandindo a capacidade em até 10×.

Estado atual em 2026. Projects agora têm memória isolada por projeto, além de um resumo separado para chats fora de projetos; a memória do histórico está disponível em Pro, Max, Team e Enterprise (web, desktop e mobile). É possível mover chats para dentro/fora de projetos para gerenciar o que entra nos resumos de memória. O conteúdo do projeto é cacheado e não conta contra os limites de uso por mensagem.

Modelo subjacente. Qualquer que seja o modelo Claude selecionado — em meados de 2026 a fronteira é o Claude Opus 4.8 (contexto de 1M de tokens na API; US\$ 5/US\$ 25 por milhão de tokens de entrada/saída), com o Sonnet 4.6 como padrão eficiente.

Contexto / memória / persistência. Upload de arquivos por chat limitado a ~20 arquivos x 30 MB cada. A persistência vem da base de conhecimento (recuperação) + instruções customizadas (sempre carregadas) + resumos de memória. Atenção: o contexto NÃO é compartilhado entre chats dentro de um projeto, a menos que seja adicionado à base de conhecimento.

Superfícies de integração. Os Projects em si são um recurso de consumidor do claude.ai; a plataforma Claude mais ampla expõe MCP (conectores remotos), integração com Google Workspace e Microsoft 365 (Word, Excel, Outlook). Artifacts renderizam código/documentos/diagramas.

Limitações. Atritos documentados: a base de conhecimento pode encher o contexto e espremer a conversa; a recuperação completa não é garantida por resposta (só os trechos mais relevantes são puxados); contextos isolados significam zero vazamento entre projetos (recurso e limitação ao mesmo tempo); limites de armazenamento; RAG só em planos pagos.

Preços. Free (5 projetos, limitado); Pro (US\$ 20/mês, US\$ 17 anual), Max 5x (US\$ 100), Max 20x (US\$ 200), Team, Enterprise — projetos ilimitados nos planos pagos.

2. Claude Code (Anthropic)

Capacidade central. Agente de codificação agêntico da Anthropic, terminal-first (GA desde maio de 2025). Lê arquivos, roda comandos de shell, navega, chama APIs, edita código, roda testes, faz commits no GitHub. Disponível em terminal, extensões VS Code/JetBrains, desktop, web e CI (um processo CLI one-shot alimenta GitHub Actions e jobs agendados).

O modelo de extensibilidade em cinco camadas (2026):

- **CLAUDE.md** — regras persistentes de projeto sempre carregadas + auto-memória (o Claude escreve as próprias notas entre sessões; requer v2.1.59+, ligado por padrão).
- **Servidores MCP** — ferramentas/dados externos; escopos local/projeto/usuário; o Tool Search carrega definições sob demanda para cortar o imposto de ~55 mil tokens de um setup típico de 5 servidores/58 ferramentas em ~85%.
- **Skills** — pacotes de instrução SKILL.md, divulgação progressiva (~100 tokens até serem invocados); a spec Agent Skills ficou aberta e funciona em Claude Code, Codex CLI, Gemini CLI, Cursor.
- **Hooks** — automação de ciclo de vida/segurança (PreToolUse para bloquear comandos inseguros).
- **Subagentes** — janelas de contexto isoladas, prompts/ferramentas próprios;

recomendados como read-only, com edições delegadas ao agente-pai.

Agent Teams (fev/2026, lançado com Opus 4.6). Várias sessões Claude autônomas que trocam mensagens entre si, reivindicam tarefas de uma lista compartilhada e contestam os achados umas das outras — padrão orquestrador-trabalhador embutido na ferramenta. O Opus 4.8 adiciona “Dynamic Workflows” (centenas de subagentes em paralelo).

Modelos subjacentes. Opus 4.8 (fronteira, contexto 1M), Sonnet 4.6 (eficiente, contexto 1M via API). Roteamento automático entre modelos por complexidade da tarefa.

Limitações. Consumo de tokens é alto. O site do Claude Code (atualizado em meados de abril de 2026, mais que dobrando a estimativa pública anterior) agora afirma que “o custo médio é de cerca de US\$ 13 por desenvolvedor por dia ativo e US\$ 150–250 por desenvolvedor por mês, com custos abaixo de US\$ 30 por dia ativo para 90% dos usuários” (os números antigos, muito citados, eram ~US\$ 6/dia em média e 90% abaixo de US\$ 12/dia). Há limites de janela móvel de 5 horas; um bug de inflação de tokens na v2.1.100 foi reportado; e a Microsoft teria revertido assentos internos por uso descontrolado (maio/2026).

Preços. Incluído em Pro (US\$ 20), Max 5x (US\$ 100), Max 20x (US\$ 200), Team Premium (~US\$ 100–125/assento, mínimo de 5 assentos), Enterprise (contexto 500K, pronto para HIPAA). Não no Free. API: Sonnet US\$ 3/US\$ 15, Opus US\$ 5/US\$ 25 por MTok.

3. Google Antigravity

Capacidade central. Plataforma de desenvolvimento agent-first do Google, lançada em 18 de novembro de 2025 junto com o Gemini 3, atualizada para **Antigravity 2.0 no Google I/O 2026 (anunciada em 19 de maio de 2026)**. Originalmente um fork do VS Code com duas superfícies — **Editor View** (IDE de IA familiar) e **Manager Surface/Agent Manager** (orquestrar múltiplos agentes de forma assíncrona entre workspaces). Os agentes operam por **editor, terminal e navegador**.

Ecossistema Antigravity 2.0:

- **Antigravity (app desktop standalone)** — agent-first, sem necessidade de IDE; centrado em projetos (agentes trabalham em múltiplas pastas); tarefas agendadas via cron; comandos de voz via áudio do Gemini; subagentes dinâmicos.
- **Antigravity IDE** — a original, ainda recomendada para desenvolvedores.
- **Antigravity CLI (agy)** — agentes de terminal; substitui o Gemini CLI (Gemini CLI/Code Assist de consumidor encerrados em 18 de junho de 2026).
- **Antigravity SDK** — construir agentes customizados; deploy na sua própria infraestrutura.

Artefatos. O mecanismo de confiança: os agentes produzem entregáveis verificáveis — planos de implementação, listas de tarefas, diffs de código, diagramas de arquitetura, screenshots e gravações de navegador (vídeo WebP) — em vez de chamadas brutas de ferramenta. Você deixa comentários estilo Google Docs nos artefatos e o agente incorpora o feedback sem parar.

Subagente de navegador. Sobe uma instância real do Chrome em perfil isolado para clicar, preencher formulários, tirar screenshots e verificar o próprio trabalho; segurança por allowlist/denylist (começa com localhost).

Conhecimento e aprendizado. "Aprendizado como primitiva central" — Knowledge Items (KIs) armazenados em `~/.gemini/antigravity/knowledge/` como markdown; um Knowledge Subagent destila conversas em KIs de forma assíncrona. AGENTS.md/GEMINI.md para regras ambientais do projeto; Skills para conhecimento procedural (escopo global `~/.gemini/skills/`, compartilhado entre os produtos Antigravity).

Modelos subjacentes. Gemini 3 Pro no lançamento → Gemini 3.1 Pro → **Gemini 3.5 Flash** (padrão do 2.0; o Google afirma que supera o Gemini 3.1 Pro em benchmarks de código/agêntico a 4× a velocidade; co-desenvolvido usando o Antigravity). Opcionalidade de modelo: também suporta **Anthropic Claude Sonnet 4.5/Opus 4.6** e **OpenAI GPT-OSS** — fato crítico para orquestração cross-model.

MCP. Painel MCP visual com marketplace (instalação por clique) mais arquivo de config (`mcp_config.json`). O acesso direto ao terminal significa que ele pode rodar outras CLIs (inclusive o Claude Code) dentro do seu terminal.

Limitações. Cloud-native (código enviado aos servidores do Google — preocupação de compliance para setores regulados); governança enterprise ainda em formação; risco de prompt-injection via navegador/documentos; status de preview público com iteração rápida.

Preços. Gratuito para indivíduos no preview público, com limites generosos do Gemini 3. AI Pro US\$ 20; novo **AI Ultra US\$ 100** (5× os limites Pro no Antigravity); o AI Ultra topo caiu para US\$ 200 (20× Pro).

4. NotebookLM (Google)

Capacidade central. Ferramenta de pesquisa ancorada em fontes: você sobe fontes e recebe respostas ancoradas em RAG, com citações, que se mantêm nos seus documentos e se recusam a responder fora deles — o que o torna uma base de conhecimento de quase-zero-alucinação. Layout de três colunas: **Fontes** (esquerda) / **Chat** (centro) / **Studio** (direita).

Estado atual em 2026:

- **Oficialmente construído sobre o Gemini 3** (confirmado em 19 de dezembro de 2025; usuários Pro/Ultra recebem Gemini 3.1 Pro), substituindo o Gemini 2.5 Flash.
- **Deep Research** — modo agêntico que cria um plano de pesquisa, navega por centenas de sites e gera um relatório estruturado e ancorado em citações que você importa (relatório + fontes) para o notebook; Fast Research para varreduras rápidas.
- **Saídas do Studio** — Audio Overviews (agora 80+ idiomas, Modo Interativo em que você “levanta a mão” e fala com os apresentadores de IA), **Video Overviews** (slides cinematográficos e narrados), Mapas Mentais, Relatórios, Guias de Estudo, Briefing Docs, FAQ, Flashcards, Quizzes, Tabelas de Dados, decks de slides (exportação PPTX), 10 estilos de infográfico. Agora dá para criar múltiplas saídas do mesmo tipo por notebook.
- **Alcance cross-tool** — montar notebooks como fontes de dados no app Gemini para consultas cross-notebook; fontes do Google Drive sincronizam automaticamente a cada poucos minutos.
- **Tipos de arquivo** — PDF, Google Sheets, .docx, imagens, EPUB, áudio; 500 mil palavras / 200 MB por fonte; 50 fontes (Free).

Modelo subjacente. Gemini 3 (Flash na maior parte; 3.1 Pro para Pro/Ultra).

Superfícies de integração. O NotebookLM de consumidor não tem API pública oficial para automação, MAS o **NotebookLM Enterprise** (parte do Gemini Enterprise, sobre Vertex AI / Discovery Engine) expõe uma API REST para criar/recuperar/listar/compartilhar notebooks. Servidores MCP da comunidade (baseados em automação de navegador) fazem a ponte do NotebookLM de consumidor para qualquer cliente MCP.

Limitações. A automação de consumidor está numa zona cinzenta dos Termos de Serviço (automação de navegador, não API oficial); não ingere YouTube/RSS nativamente; Modo Interativo só em inglês; o Deep Research retorna algumas fontes inúteis (com paywall); rollouts faseados deixam a disponibilidade de recursos desigual.

Preços. 4 camadas: Free, Plus, Pro, Ultra; incluído nas assinaturas Google AI e no Gemini Enterprise (Standard/Plus).

Parte 2 — Arquiteturas de Orquestração Disruptivas

O tecido conectivo: MCP como barramento cross-vendor

O MCP (Model Context Protocol) foi doado em 9 de dezembro de 2025 à Agentic AI Foundation (fundo dirigido sob a Linux Foundation, co-fundado com Block e OpenAI).

Segundo a Linux Foundation, há agora “mais de 10.000 servidores MCP publicados... cobrindo de ferramentas de desenvolvedor a deployments Fortune 500”, e o projeto MCP reporta “mais de 97 milhões de downloads mensais de SDK, 10.000 servidores ativos”. As quatro ferramentas tocam nele: o Claude Code consome e expõe MCP; o Antigravity tem marketplace MCP visual + `mcp_config.json`; o NotebookLM é alcançável via servidores MCP da comunidade; e a plataforma-mãe do Claude Projects suporta conectores MCP remotos. É isso que torna um sistema de quatro ferramentas construível **hoje**.

Pontes documentadas que já existem:

- **NotebookLM ↔ Claude** — `jacob-bd/notebooklm-mcp-cli` e `PleasePrompt/notebooklm-mcp` expõem 16–35 ferramentas (perguntar, adicionar fontes, gerar artefatos do Studio, baixar). A CLI até tem `nlm setup add antigavity` e `nlm setup add claude-code`, então a MESMA camada de conhecimento do NotebookLM se conecta aos dois fornecedores. Fluxo real: o Claude Code pergunta ao NotebookLM “como funciona a API X?” e recebe respostas ancoradas e com citações — eliminando APIs alucinadas e mantendo baixo o gasto de tokens do Claude (só a resposta sintetizada trafega).
- **Antigravity ↔ Claude Code** — o servidor MCP `anderson-suga/claude-advisor` faz do Antigravity o orquestrador estratégico e do Claude Code o executor/validador; o `SinanTufekci/Claude-Code-Antigravity-CLI-MCP-Server` faz o inverso, expondo a CLI `agy` do Antigravity (Gemini 3.5 Flash) como subagente para o Claude Code, para segundas opiniões baratas. Você também pode instalar a extensão do Claude Code dentro do próprio Antigravity (“Spark”).

Arquitetura A — O Pipeline de Build Ancorado em Conhecimento (mais prático)

NotebookLM (Gemini 3) → Claude Projects → Claude Code (Opus 4.8) → Antigravity (G
oráculo de ancoragem memória estratégica executor preciso verificado

1. **NotebookLM como camada de pesquisa/ancoragem.** Rode o Deep Research para navegar por centenas de sites → importe relatório + fontes. Carregue docs de API, specs, papers (até 300 PDFs reportados na prática). Isso vira um oráculo de zero-alucinação.
2. **Claude Projects como estratégia durável.** Destile o briefing do NotebookLM em instruções de projeto + base de conhecimento. Essa é a “estrela-guia” persistente que sobrevive a todas as sessões do Claude Code.
3. **Claude Code como executor.** Com o MCP do NotebookLM acoplado, o Claude Code

consulta o oráculo sempre que tem dúvida (em vez de alucinar), e então implementa com a precisão do Opus 4.8. O CLAUDE.md carrega as regras do projeto espelhadas do Claude Projects.

4. **Antigravity como orquestrador paralelo + verificador.** Passe o bastão para a Manager Surface do Antigravity para rodar múltiplos agentes em paralelo e, crucialmente, use o **subagente de navegador para verificar** que o build de fato funciona (cliques, screenshots, gravações como Artefatos). O Gemini 3.5 Flash faz isso rápido e barato.

Por que é disruptivo: cada ferramenta cobre a fraqueza da outra — o NotebookLM mata a alucinação, o Claude Projects mata a perda de contexto, o Claude Code dá precisão, o Antigravity dá paralelismo + verificação empírica. Nenhuma ferramenta sozinha faz as quatro coisas.

Arquitetura B — O Conselho Cross-Model (o “conselho de agentes”)

Este é o núcleo disruptivo para um fundador construindo um sistema de deliberação. O padrão se apoia em três precedentes reais:

- **Sistema de pesquisa orquestrador-trabalhador da Anthropic:** o agente líder (Opus) decompõe, gera subagentes paralelos (Sonnet) com janelas de contexto próprias, escreve o plano na memória antes do truncamento de 200k tokens, e os subagentes retornam artefatos condensados via sistema de arquivos compartilhado. Segundo o post de engenharia da Anthropic “How we built our multi-agent research system”, “um sistema multi-agente com Claude Opus 4 como agente líder e subagentes Claude Sonnet 4 superou o Claude Opus 4 single-agent em 90,2% na nossa avaliação interna de pesquisa”. O custo: “os agentes tipicamente usam ~4x mais tokens que interações de chat, e sistemas multi-agente usam ~15x mais tokens que chats”. A Anthropic também descobriu que “o uso de tokens por si só explica 80% da variância” no desempenho (com número de chamadas de ferramenta e escolha de modelo como os outros fatores, somando 95% juntos).
- **LLM Council de Karpathy:** despacha uma consulta para múltiplos modelos de fronteira via OpenRouter → os modelos rankeiam anonimamente as respostas uns dos outros → um LLM “Presidente” sintetiza. Segundo o README do GitHub, “Este projeto foi 99% vibe coded como um hack divertido de sábado porque eu queria explorar e avaliar vários LLMs lado a lado”. Os membros padrão são “OpenAI GPT 5.1, Google Gemini 3.0 Pro, Anthropic Claude Sonnet 4.5, xAI Grok 4”. A revisão cruzada entre modelos frequentemente escolhe a resposta de outro modelo como superior.
- **Skills `agent-council` / `llm-council` do Claude Code:** convocam Claude Code + Codex CLI + Gemini CLI; aquela de onde você invoca é o “presidente”, as outras são

membros; deliberação em três estágios (resposta independente → revisão por pares → síntese).

A instanciação do conselho de quatro ferramentas:

- **Manager Surface do Antigravity** = a câmara/orquestrador do conselho (pode rodar Gemini, Claude e GPT-OSS nativamente — diversidade cross-model verdadeira numa só superfície).
- **Claude Code (Opus 4.8)** = o membro de precisão do conselho + presidente para decisões de código.
- **Um membro Gemini** via `agy` do Antigravity (Gemini 3.5 Flash) = perspectiva divergente rápida.
- **NotebookLM** = o “secretário/checador de fatos” sem voto — toda afirmação que um membro do conselho faz é validada contra fontes ancoradas antes de o presidente sintetizar. Este é o movimento genuinamente novo: a maioria dos conselhos não tem membro de ancoragem, então podem estar confiantemente errados em uníssono. Adicionar o NotebookLM como árbitro de ancoragem ataca diretamente a **alucinação correlacionada**.

A pesquisa acadêmica “Council Mode” (arXiv:2604.02923, “Council Mode: A Heterogeneous Multi-Agent Consensus Framework”) reporta “uma redução relativa de 35,9% nas taxas de alucinação no HaluEval, uma melhora de 7,8 pontos no TruthfulQA e uma redução de 85–89% na variância de viés entre domínios” (alucinação média de 10,7% vs 16,7% do melhor modelo individual, Claude Opus 4.6) — evidência de que o prêmio de diversidade é real. O trade-off: o mesmo paper reporta latência “aproximadamente 2–3× comparada a modelos individuais”, então o conselho só deve disparar em decisões de arquitetura/estratégia de alto risco.

Arquitetura C — O Loop de Sincronização de Memória Persistente

O problema difícil entre as quatro ferramentas é a **memória persistente compartilhada**. Cada uma tem seu próprio repositório: Claude Projects (base de conhecimento + resumos de memória), Claude Code (CLAUDE.md + auto-memória em markdown), Antigravity (Knowledge Items em markdown em `~/gemini/antigravity/knowledge/`), NotebookLM (fontes).

O padrão de sincronização disruptivo: fazer do markdown a língua franca. KIs do Antigravity, auto-memória do Claude Code e CLAUDE.md são todos markdown puro; o NotebookLM ingere markdown/docx e sincroniza do Drive a cada poucos minutos. Então:

1. Designe o NotebookLM (ou uma pasta do Drive) como o **repositório canônico de conhecimento**.

2. Claude Code e Antigravity escrevem seus aprendizados como markdown nesse repositório (KIs, auto-memória).
3. O NotebookLM sincroniza automaticamente e re-ancora — fechando o loop, de modo que toda ferramenta lê de uma única fonte de verdade em evolução.
4. A base de conhecimento do Claude Projects é atualizada periodicamente a partir do mesmo repositório.

Isso transforma quatro memórias isoladas em um único “segundo cérebro” compartilhado, que se acumula entre sessões e entre famílias de modelos.

Arquitetura D — O “Turno da Noite” Assíncrono

Combine as **tarefas agendadas via cron** do Antigravity 2.0 + os **jobs CLI one-shot/agendados** do Claude Code + o **Deep Research em background** do NotebookLM: agende execuções noturnas em que o NotebookLM atualiza a base de conhecimento via Deep Research, os agentes do Antigravity implementam e verificam features via navegador em paralelo, e o Claude Code roda subagentes de teste/revisão — todos reportando de volta como Artefatos e PRs que você revisa de manhã. É o padrão “acordar com o trabalho pronto e verificado”, agora viável porque os três suportam execução agendada não-assistida.

Referências de Vídeos de Alto Alcance e Fontes Autoritativas

(As contagens de visualizações abaixo são as melhores estimativas disponíveis; a proteção anti-bot do YouTube impediu verificação ao vivo exata — trate como aproximadas e reconfirme antes de publicar.)

- **“Learn the basics of Google Antigravity”** — canal oficial do Google Antigravity, ~1,39M de views reportadas (youtube.com/watch?v=nTOVIGsqCuY). O primer oficial mais assistido.
- **“Google Antigravity | I/O 2026 Keynote”** (demo ao vivo de Varun Mohan) — oficial do Google (youtube.com/watch?v=T_fnhr5lVBw).
- **“Mastering Claude Code in 30 minutes”** — oficial da Anthropic, apresentador Boris Cherny (criador do Claude Code), ~1,3M de views reportadas (youtube.com/watch?v=6eBSHbLKuN0). O tutorial canônico do Claude Code.
- **“Learn 80% of NotebookLM in Under 13 Minutes!”** — Jeff Su, seu tutorial-carro-chefe de NotebookLM cobrindo Audio Overviews (youtube.com/watch?v=EOmgC3-hznM).

- **LLM Council** — Andrej Karpathy lançou como repositório open-source no GitHub (github.com/karpathy/llm-council), descrito no README como "99% vibe coded como um hack divertido de sábado"; não há vídeo oficial do Karpathy, então cite o repo como fonte primária.

Documentação primária: Anthropic (anthropic.com , o blog de engenharia do sistema multi-agente, docs do Claude Code em code.claude.com , support.claude.com); Google (antigravity.google/blog , posts do NotebookLM em blog.google , developers.googleblog.com , docs do NotebookLM Enterprise no Google Cloud, Google Codelabs).

Recomendações

Estágio 1 — Provar o loop de ancoragem (semana 1). Instale um servidor MCP do NotebookLM ([jacob-bd/notebooklm-mcp-cli](https://github.com/jacob-bd/notebooklm-mcp-cli)) e conecte-o ao Claude Code. Carregue seus docs de domínio num notebook. Valide que o Claude Code para de alucinar APIs consultando o oráculo. Este é o movimento de maior ROI e menor esforço. **Benchmark para avançar:** queda mensurável em chamadas de API erradas/alucinadas e menor gasto de tokens do Claude por tarefa.

Estágio 2 — Adicionar a divisão executor/orquestrador (semanas 2–3). Conecte Antigravity ↔ Claude Code via a ponte `claude-advisor` (Antigravity orquestra, Claude executa) ou a ponte `agy` (Claude delega trabalho barato ao Gemini Flash). Use o subagente de navegador do Antigravity para verificação empírica. **Benchmark:** o Gemini-Flash lidando com >50% do trabalho de baixo risco a uma fração do custo de tokens do Claude, com artefatos de verificação em todo build.

Estágio 3 — Montar o conselho, seletivamente (semanas 3–4). Implemente a Arquitetura B só para decisões de alto risco, usando o NotebookLM como árbitro de ancoragem. Comece com a skill `agent-council` , depois customize. **Benchmark / limiar:** só invoque o conselho quando o valor da decisão claramente exceder o prêmio de ~2–3× de latência e ~15× de tokens; se as saídas do conselho raramente divergirem do Opus single-model, rebaixe para single-model nessa classe de tarefa.

Estágio 4 — Unificar memória e automatizar (mês 2). Implemente a Arquitetura C (markdown-como-língua-franca sincronizado por um repositório canônico Drive/NotebookLM) e a Arquitetura D (execuções de turno noturno agendadas). **Limiar para escalar:** quando a deriva de memória compartilhada (ferramentas agindo sobre contexto desatualizado) cair a quase zero ao longo de uma semana de execuções.

Regra de bolso para alocação de família de modelos: Opus 4.8 para

precisão/profundidade e síntese final; Gemini 3.5 Flash para abrangência, paralelismo e verificação; NotebookLM/Gemini 3 para ancoragem; reserve o conselho cross-model completo para decisões em que a falha correlacionada seja cara.

Ressalvas

- **O custo é a restrição dominante.** Sistemas multi-agente “usam ~15x mais tokens que chats” (Anthropic); conselhos cross-model somam ~2–3x de latência (paper Council Mode). O próprio site do Claude Code agora coloca a média em ~US\$ 13/dev/dia ativo e US\$ 150–250/dev/mês, com 90% abaixo de US\$ 30/dia ativo — e power users reportam bem mais. A Microsoft teria revertido assentos internos de Claude Code por uso descontrolado. Faça orçamento e imponha tetos agressivos.
- **As pontes cross-vendor são não-oficiais.** O MCP de consumidor do NotebookLM usa automação de navegador numa zona cinzenta de ToS; a ponte `agy` lê arquivos de estado não documentados do Antigravity e pode quebrar em atualizações. Para produção/compliance, use a API REST oficial do NotebookLM **Enterprise** e trate as pontes da comunidade como experimentais.
- **Essas ferramentas mudam toda semana.** Versões de modelo, preços, cotas e recursos (ex.: encerramento do Gemini CLI em 18/06/2026; Antigravity ainda em preview público) mudam constantemente. Reverifique os detalhes antes de comprometer a arquitetura.
- **Cloud/compliance.** O Antigravity é cloud-native (código para servidores do Google); o NotebookLM de consumidor difere do Enterprise no tratamento de dados. Setores regulados devem usar as camadas Enterprise por padrão.
- **A falha correlacionada não está totalmente resolvida.** Conselhos reduzem mas não eliminam a alucinação compartilhada; agentes a jusante podem amplificar um erro a montante. O padrão NotebookLM-como-árbitro mitiga isso, mas depende da qualidade das fontes ancoradas.
- **Contagens de views são aproximadas** — a verificação automatizada do YouTube foi bloqueada; confirme antes de citar números exatos.